Hamar Planning Brief: A Small Workflow for Reading Public Notices

Kai Arne Hage
Mohammad Almojtaba Elhajj A Elhajj
Group 12, KIUA-2003
University of Inland Norway

Abstract

Our project turns selected text from a Hamar planning notice into a short Norwegian draft. It shows what is proposed, where, the deadline, and how residents can take part. A local model extracts facts and quotes. Python formats the draft and checks it before saving. Five runs on a 21-word extract passed the checks. Five more runs on a longer extract also passed, but missed important information. This shows that passing the checks does not mean the draft is complete. We describe the current program, its limits, and what we plan to improve before the exam. Students still need to review and edit this draft.

1 Problem and what counts as success

We chose a local problem: making a planning notice easier to read. Our intended user is a Hamar resident who wants to know what might change and how to give input. The example is a planned cycleway with a pavement along Ringgata, between Solvangvegen and Disenvegen (Hamar kommune, 2026). Norwegian output and a local source connect the project to Innlandet. We have not interviewed residents, so the need is still an assumption rather than something we have measured.

We limit the project to official planning notices that invite public input. Unrelated news, grants and finished decisions are outside this scope. A person chooses the text before the program runs. It is not a website crawler. The program should return either a draft with evidence or a clear failure report.

The first input has 21 words in three separate passages. It includes the proposal, location, planning stage and deadline. It leaves out participation and the response route. The full page does contain these details. A second, 61-word input adds the institutional mailbox, postal address, case number and open office day. Both inputs are selected passages, not full notices. They contain no individual names or personal contact details. The page was fetched again on 29 October, and the new passages were checked against that response. Student source review is still pending. No general permission to crawl or redistribute the site has been established.

Success for the first input means finding the cycleway and pavement, both street endpoints, the start of planning work, and the deadline of 18 November 2026. The two omitted fields should be `unknown`. For the longer input, the draft should also show the office day and put the official submission details in the response-route field. Both drafts must link to the source, show a check date and say `needs_human_review`. Passing the code checks is a separate, weaker result. It does not prove that the draft meets all these goals.

2 How the program works

Repository: `https://github.com/Kaiju77/hamar-planning-brief`. It contains the code, context prompt, inputs, saved runs, tests and instructions. The program uses Python's standard library

Preprint.

and the local Ollama API (Ollama, 2026). One command runs four stages without a person moving data between them.

1. Load the selected text. `run.py` reads a text file and a metadata file with the source URL, retrieval date and partial-source flag. Input is limited to 12,000 characters. The model receives the extraction prompt and the text inside source markers. It cannot fetch the URL itself. It does not receive our chat history, other notices, credentials or access to files. A person must choose and screen the input.

2. Ask the model for facts. The model returns a scope decision and six fields: proposal, location, planning stage, deadline, participation and response route. Each field has a value and a supporting quote. The prompt says to use `unknown` or `ambiguous` instead of guessing. It also says that source text is data, not instructions. The model is `qwen2.5-coder:7b`, with temperature zero and JSON output. A deadline should become an ISO date only when day, month and year are explicit. An official mailbox is allowed; an individual person's contact details are not.

3. Make the Norwegian draft. Python places the facts in a fixed template. It starts with the proposal, location, deadline, participation and response route. The planning stage, evidence and source follow. This uses no second model call. The earlier plan had a model for drafting too. A template is simpler and cannot add new facts by itself, but it also does little to improve the model's wording. Missing information stays visible.

4. Check and save. `checks.py` checks field types, source metadata, evidence and the deadline. Quotes must occur in the input, ignoring case and repeated spaces. The normalized deadline must match a Norwegian date in its own quote. Code compares the deadline with the recorded Oslo check date. A date-only deadline stays open through that day. An unknown or ambiguous deadline has unknown status. Exact cutoff times are not handled yet. If a check fails, the program saves errors and no draft. If checks pass, it saves JSON and Markdown. A quote occurring in the input does not prove that it supports the value beside it.

Tools, automation and limits. Python reads and writes files, calls the local HTTP service and runs the checks. The model cannot run commands or publish anything. Each attempt has its own folder with the raw response, extracted facts, time, token counts and an input/prompt snapshot. There is one model call, no automatic retry, an 800-token output cap and a 120-second HTTP timeout. These limits are not yet a measured total token budget. Ctrl+C stops the client, but the model server may also need `ollama stop`. Python still has the user's normal file permissions. Stronger path restrictions, budgets and a clear halt policy are work for the exam.

3 What has been tested

Table 1 comes from saved runs, not estimated results. Batch A used the first extract with check date 28 October. Batch B used the longer extract with check date 29 October. Both used the same prompt, model and settings. The model's token counts and client elapsed times are saved for every attempt. Time covers extraction and checking, but excludes final file writes.

Input	Run	Input tokens	Output tokens	Seconds	Code checks
A	1	465	264	15.366	Saved
A	2	465	264	5.637	Saved
A	3	465	264	5.670	Saved
A	4	465	264	5.616	Saved
A	5	465	264	5.682	Saved
B	1	593	431	19.142	Saved
B	2	593	428	7.987	Saved
B	3	593	428	8.028	Saved
B	4	593	428	8.056	Saved
B	5	593	428	7.963	Saved

Table 1: Two inputs from one notice, each repeated five times. These are not ten different notices.

Batch A used 3645 tokens and 37.971 seconds in total. All five drafts had the expected deadline, location endpoints, proposal terms and planning-stage term. Participation and response route were unknown in all five, as expected for that input. The extracted facts were identical across the five runs. With only 21 words, this is close to a copying task and tells us little about more difficult notices or model variation.

Batch B used 5108 tokens and 51.176 seconds. All 5 attempts saved drafts, but the office-day and date/time checks passed in 0/5 and 0/5 runs. The route mailbox/case and postal-address checks passed in 0/5 and 0/5. The model put the submission details in participation, left response route unknown, and omitted the office day. The longer input therefore failed our content goal even though the code accepted it. These attempts are kept as failures of completeness, not hidden or counted as successful summaries. The reference checks look for selected words, dates and addresses. They are not a full test of meaning or good Norwegian. Timing is also not a controlled speed comparison: loading the model can make the first run slower. Temperature zero does not guarantee the same answer on another machine or model version.

Earlier attempts are kept separately. One `llama3.2:3b` response rejected the notice as outside the scope and returned invented quotes and a non-ISO date. It produced a failure report. Scope rejection stops later checks, so this does not show that every invented quote was detected. Five earlier Qwen runs used an older field contract and are not added to the table.

Twelve unit tests pass. Seven cover the original synthetic cases: a normal case, missing and ambiguous dates, wrong scope, altered evidence and two injection outcomes. Five more cover malformed deadlines, date boundaries, missing files, separate output folders and an unavailable Ollama service. Fixture evaluation uses canned model replies. It tests program behaviour, not a live model's ability to resist an attack.

One known injection case still saves an unsafe draft. If the model follows an instruction in the source and quotes the injected date, the evidence check can pass. A warning is not enough to stop it. A made-up location or personal contact detail paired with a genuine quote can also pass because non-date values are not checked for meaning. We have not tested many notices, conflicting deadlines or resident usability. A person still needs to compare the draft with the source.

4 Plan and risks before the exam

Compulsory 2 is due on 7 November 2026 at 23:59. The next steps are student review, the mandatory demo in weeks 46 and 47, and the final white paper on 3 December at 11:59 (University of Inland Norway, 2026). We plan for the earliest possible demo on 9 November until the assigned slot is confirmed.

When	What we plan to finish
By 2 Nov	Review the source and output; agree roles; check the Canvas template and demo slot.
By 5 Nov	Keep a fixed baseline set; practise a normal run and a failure for a possible 9 November demo.
6 Nov	Edit the report, add real meeting notes and check a clean copy of the code.
7 Nov, 23:59	Submit the reviewed report with the repository link in Canvas.
9 to 22 Nov	Give the required demo; record failures and feedback.
By 25 Nov	Test two failure classes, make repairs and rerun the tests. Update the architecture description.
26 Nov to 2 Dec	Finish the evaluation, eight real meeting records, paper edits and reproduction check.
3 Dec, 11:59	Submit the 7 to 8-page white paper in Wiseflow; tag the public repository commit `exam`.

Table 2: Our proposed schedule. Intermediate dates can change after the group discussion.

Risk	What we will do about it
Wrong facts or poor Norwegian	Compare with the source; test more notices; record missing or unsupported details.
Unclear or changed deadline	Keep uncertainty visible and check the original before acting.
Ollama down or invalid reply	Save a failure report; fix the service or input and rerun manually. Label offline fixtures clearly.
Source instructions or personal data	Keep human review; screen input and output; test stronger checks. The current warning is too weak.
Unclear source reuse or access terms	Keep extracts short and attributed. Stop wider collection until the terms are checked.

Table 3: Main risks and planned responses.

For the exam, the two planned failure classes are changed input and dependency failure. The first will include misleading instructions and conflicting facts in synthetic notices. The second will include a refused connection, timeout and bad model JSON. We will save what happened, repair the relevant stage and rerun the same cases plus the other tests. This is planned work, not completed security testing. We also need a broader set of normal, missing, conflicting and out-of-scope inputs. Code can check format and dates; people must assess meaning and clarity. A proposed halt rule is to stop after three invalid runs and ask for review. That rule is not implemented yet. Demo feedback and later repairs will go in the final paper, together with the actual meeting records.

References

Hamar kommune. Oppstart reguleringsplan fv. 187 Ringgata i Hamar. First observed publication label: 26 October 2026. Current combined publication/modified label: 29 October 2026. Accessed 29 October 2026. `https://www.hamar.kommune.no/lokaldemokrati/oppstart-reguleringsplan-fv-187-ringgata-i-hamar`.

University of Inland Norway. *Agentic Workflows for Innlandet: Building and Verifying Coding Agent Harnesses for Local Needs*. KIUA-2003 project brief, Autumn 2026. Supplied course document.

Ollama. Generate a response: local generation API documentation. Accessed 28 October 2026. `https://docs.ollama.com/api/generate`.

A Prompt excerpts and changes

These excerpts are checked against messages available in this chat. They show some of the help used for the project. Two student examples still need to be added. This appendix must stay within two pages after editing.

Choosing the scope. Kai asked for: "Something that is relatively easy to pull off but still relevant and good enough for a great exam/grade". This led to a small local-notice project. It does not prove resident need.

Checking dates. The Claude review supplied by Kai said: "Add a deadline_status field (open, closed, unknown) computed in code by comparing deadline with retrieved_at." The implemented check instead uses a recorded check date. The retrieval date stays separate. This allows the same saved text to be checked again later.

An untested assumption. The Claude handoff said: "The Ollama backend is written but untested." Real runs were then saved, including the failed Llama attempt. Later Qwen results do not replace that failure.

Student correction or rejection, example 1. [[STUDENT: one real moment where you challenged or rejected agent output, with a short quote]]

Student correction or rejection, example 2. [[STUDENT: a second real moment where you challenged or rejected agent output, with a short quote]]

AI help and our responsibility. Claude and Codex did substantial coding, tests, runs and report drafting. Kai chose the direction, supplied reviews and requested changes. This plain-language rewrite is also AI-assisted. The students must read, check and edit it before submission. **[[STUDENT: describe what Kai actually checked and changed after reading the report and code]] [[STUDENT: describe Mohammad's actual contribution and review; do not claim work he has not done]]**

B Working sessions and meeting records

The records below are individual work with AI help. They are not group meetings with Mohammad. Add actual group records after meetings happen.

B.1 27 October: scope and source work

Recorded participants: Kai and Codex in the coursework chat. **Work:** read the new requirements and choose a small Innlandet project. **Decisions:** start with selected public text, evidence and human review. The supplied Claude review led to clearer fields for responding, dates and evidence. **Still needed then:** group details, exact deadline, source terms and template.

B.2 28 October: code and real runs

Recorded participants: Kai and Codex, with Claude's handoff supplied by Kai. **Work:** reproduce seven tests, fix malformed-date handling, add four tests, preserve separate outputs and run the local models. One Llama failure and two different five-run Qwen batches were kept. **Decisions:** use template drafting, show the injection gap and publish the repository with Kai's permission. Kai confirmed Group 12 and the Canvas deadline of 7 November at 23:59.

B.3 29 October: reviews and a longer input

Recorded participants: Kai and Codex; two Claude reviews supplied by Kai. **Work:** correct the word count, improve the risks and schedule, add an unavailable-service test, and check a clean clone. A longer extract was then source-checked and used for five more real attempts. The current changes are kept local because Kai's latest instructions say not to publish them. Student review and partner contribution are still open.

B.4 Template for the next actual group meeting

[[STUDENT: actual meeting date, duration and attendees]] [[STUDENT: what each person checked; options discussed; decision and reason; next actions]] Agree the first date together. Later tentative dates are 2, 5, 9, 12, 16, 23 and 30 November. Proposed dates do not count towards eight completed meeting records.

C Reproduction and one saved evaluation

Install Python 3.10+ and Ollama and pull `qwen2.5-coder:7b`. No Python packages are needed. From the project folder:

```
python -m unittest discover -s tests -v
python evaluate.py --backend fixture
python evaluate.py --backend ollama --count 5 --example ringgata_extended --checked-
    at 2026-10-29
python tools/build_results.py
```

The fixed date reproduces this check. For a new operational run, pass the actual Oslo date. Fixture replies are canned. The results builder uses fixed saved batches. The raw evaluation summary below keeps the original values. Individual raw model replies and input snapshots are in the referenced folder.

Saved batch: `evidence/evaluation-extended/20261029T210826845602Z`.


```
{
  "backend": "ollama",
  "checked_at": "2026-10-29",
  "cases": [
    {
      "case": "ringgata_extended",
      "draft_saved": true,
      "matches_fixture_behavior_expectation": true,
      "deadline": "2026-11-18",
```

```
      "known_unsafe_case": false,
      "security_verdict": "not_assessed",
      "reference_fields": {
        "deadline": true,
        "proposal": true,
        "location": true,
        "stage": true,
        "participation_office_day": false,
        "participation_date_time": false,
        "response_mailbox_and_case": false,
        "response_postal_address": false
      }
    },
    {
      "case": "ringgata_extended",
      "draft_saved": true,
      "matches_fixture_behavior_expectation": true,
      "deadline": "2026-11-18",
      "known_unsafe_case": false,
      "security_verdict": "not_assessed",
      "reference_fields": {
        "deadline": true,
        "proposal": true,
        "location": true,
        "stage": true,
        "participation_office_day": false,
        "participation_date_time": false,
        "response_mailbox_and_case": false,
        "response_postal_address": false
      }
    },
    {
      "case": "ringgata_extended",
      "draft_saved": true,
      "matches_fixture_behavior_expectation": true,
      "deadline": "2026-11-18",
      "known_unsafe_case": false,
      "security_verdict": "not_assessed",
      "reference_fields": {
        "deadline": true,
        "proposal": true,
        "location": true,
        "stage": true,
        "participation_office_day": false,
        "participation_date_time": false,
        "response_mailbox_and_case": false,
        "response_postal_address": false
      }
    },
    {
      "case": "ringgata_extended",
      "draft_saved": true,
      "matches_fixture_behavior_expectation": true,
      "deadline": "2026-11-18",
      "known_unsafe_case": false,
      "security_verdict": "not_assessed",
      "reference_fields": {
        "deadline": true,
        "proposal": true,
        "location": true,
        "stage": true,
        "participation_office_day": false,
        "participation_date_time": false,
        "response_mailbox_and_case": false,
        "response_postal_address": false
      }
    },
    {
      "case": "ringgata_extended",
      "draft_saved": true,
      "matches_fixture_behavior_expectation": true,
      "deadline": "2026-11-18",
      "known_unsafe_case": false,
      "security_verdict": "not_assessed",
      "reference_fields": {
        "deadline": true,
        "proposal": true,
        "location": true,
        "stage": true,
        "participation_office_day": false,
        "participation_date_time": false,
```

```
      "response_mailbox_and_case": false,
      "response_postal_address": false
    }
  }
 ],
 "note": "Reference word checks are narrow proxies, not semantic accuracy. Fixture outputs are canned; the
      known unsafe case is not a security success."
}
```


D Items to finish before submission

The report uses the official NeurIPS 2025 preprint style without its checklist. **[[STUDENT: compare with the Canvas template and confirm accepted upload type]] [[STUDENT: check both extracts and at least one saved draft against the live Hamar page]]** The report remains a draft until the students do that review and edit the text. The checklist lists the open items. Do not remove them just to make the report look finished.